Pipeline Core Language

执行类别、异步操作与同步语义

1 静态对象与执行类别

令 Pipe、Addr、Lock 分别表示 pipeline、存储单元地址和锁的标识集合。每个地址标识一个最小原子访问单元；有限地址集合表示一次区域访问。共享内存记录为 $\mu : \mathsf{Addr} \rightharpoonup \mathsf{Val}$。两个访问使用相同地址时访问同一存储单元。源程序中的不同指针若存在别名，翻译时必须保留它们所指向的地址相等关系。

执行类别是静态信息的一部分：

$$\mathsf{ExecKind} = \{\mathsf{Cube}, \mathsf{Vector}, \mathsf{Scalar}, \mathsf{MTE}\}, \qquad \mathrm{kind} : \mathsf{Pipe} \to \mathsf{ExecKind}.$$

pipeline 标识包含所属设备、core 和具体执行队列的身份。例如 $p_{c,\mathrm{V}}$ 与 $p_{c',\mathrm{V}}$ 在 $c \neq c'$ 时具有不同标识，即使它们的执行类别都是 Vector。

一个程序使用有限 pipeline 集合

$$\mathcal{P} \subseteq \mathsf{Pipe}.$$

每个 $p \in \mathcal{P}$ 具有自己的顺序控制状态和局部环境。程序表示已经完成 pipeline 分配后的代码；源程序向多个 pipeline 分发指令的控制依赖需要在翻译中保留。

静态环境 Γ 包含 $\mathcal{P}$, kind、值类型、纯函数签名、事件声明、锁声明，以及以下能力谓词：

$$\mathrm{Supports}(p, f, \bar{\tau}) \in \mathsf{Bool},$$
$$\mathrm{CanRead}(p, a) \in \mathsf{Bool},$$
$$\mathrm{CanWrite}(p, a) \in \mathsf{Bool}.$$

Supports 描述 pipeline 支持的函数及输入类型，类型可以携带向量或矩阵的形状。读写权限独立指定。Cube/Vector 的执行类别由 kind 确定；地址用于记录存储位置和访问冲突。

数学函数 f 具有完整类型

$$f : D_f \longrightarrow \mathsf{Val}_{\tau}, \qquad D_f \subseteq \mathsf{Val}_{\tau_1} \times \cdots \times \mathsf{Val}_{\tau_m}.$$

f 在 D_f 上是确定且终止的纯函数。数值精度、舍入方式和形状约束属于具体函数的定义。例子使用整数算术。同一个数学函数可以由多个执行类别支持；执行类别标注确定当前操作使用的执行单元。它的数学结果由 f 确定。

事件具有完整标识

$$\varepsilon = (s, p, q, k) \in \mathsf{Event},$$

其中 s 是同步域，p 是源 pipeline，q 是目标 pipeline，k 是事件编号。每个事件声明要求 $p, q \in \mathrm{Participants}(s)$，并定义

$$\mathrm{MaySet}(p', \varepsilon) \iff p' = p,$$
$$\mathrm{MayWait}(p', \varepsilon) \iff p' = q.$$

事件状态采用二值模型。每把锁 ℓ 声明一个允许参与的 pipeline 集合 $\mathrm{Participants}(\ell) \subseteq \mathcal{P}$。同步域在本模型中限制参与者；内存本身采用共享存储单元上的交错访问语义。

2 语法与局部值

普通变量 $x \in \mathsf{Var}$ 与句柄变量 $h \in \mathsf{HVar}$ 属于不相交的集合。普通值包括整数、布尔值、地址、有限地址集合、有限地址值映射、向量、矩阵和 unit。句柄只能通过专门的句柄变量使用。

$$\begin{aligned}
e &::= v \mid x \mid g(e_1, \ldots, e_m),\\
o &::= \mathsf{read}(e_R) \mid \mathsf{write}(e_R, e_V) \mid \mathsf{computation}_{k,f}(e_1, \ldots, e_m),\\
a &::= x := e \mid h := \mathsf{begin}(o) \mid x := \mathsf{end}(h)\\
&\quad \mid \mathsf{setflag}(\varepsilon) \mid \mathsf{waitflag}(\varepsilon) \mid \mathsf{lock}(\ell) \mid \mathsf{unlock}(\ell),\\
C &::= \mathsf{skip} \mid a \mid C; C\\
&\quad \mid \mathsf{if}\ e\ \mathsf{then}\ C\ \mathsf{else}\ C \mid \mathsf{while}\ e\ \mathsf{do}\ C,\\
P &::= \mathsf{parallel}\{p_1 : C_1, \ldots, p_n : C_n\}.
\end{aligned}$$

程序中的 pipeline 标识互不相同，并确定 $\mathcal{P} = \{p_1, \ldots, p_n\}$。语法中的 $k \in \mathsf{ExecKind}$ 是计算操作的执行类别。具体程序写法使用 `computation[Vector,add](...)` 和 `computation[Cube,matmul](...)`。

表达式按局部环境求值：

$$[\![e]\!]_\rho = v.$$

g 是带有明确定义的纯表达式运算，包含算术、比较、有限映射构造及索引等。表达式不执行共享内存访问。e_R 求值得到有限区域 R；e_V 求值得到 $V : R \to \mathsf{Val}$。区域读取返回按地址索引的有限映射 B。例如 $B[a]$ 是读取结果中地址 a 对应的值。

$\mathsf{computation}$ 的输入在启动时求值并保存。任务的数学计算满足

$$\mathrm{ReadSet}(\mathsf{computation}_{k,f}) = \varnothing, \qquad \mathrm{WriteSet}(\mathsf{computation}_{k,f}) = \varnothing.$$

它也不操作事件与锁。需要从内存取得输入或保存输出时，程序显式使用 read 与 write。

begin 启动任务并继续执行后续命令；后台规则推进任务；end 等待任务完成并取得返回值。同步执行形式是以下语法糖：

$$\begin{aligned}
x := \mathsf{read}(R) &\triangleq h := \mathsf{begin}(\mathsf{read}(R));\ x := \mathsf{end}(h),\\
\mathsf{write}(R, V) &\triangleq h := \mathsf{begin}(\mathsf{write}(R, V));\ u := \mathsf{end}(h),\\
x := \mathsf{computation}_{k,f}(\bar{e}) &\triangleq h := \mathsf{begin}(\mathsf{computation}_{k,f}(\bar{e}));\ x := \mathsf{end}(h).
\end{aligned}$$

这里 h, u 是新鲜局部变量。语法糖中的区域读取仍然返回有限映射。

3 执行状态与操作句柄

全局状态定义为

$$\Sigma = \langle \mu, F, L, Q, n, T \rangle, \qquad T(p) = (C_p, \rho_p, \eta_p), \qquad p \in \mathcal{P}.$$

各分量的含义和类型如下。

分量	类型	含义
μ	$\mathsf{Addr} \rightharpoonup \mathsf{Val}$	共享内存中的地址和值
F	$\mathsf{Event} \to \{0,1\}$	二值事件标志
L	$\mathsf{Lock} \to (\mathcal{P} \cup \{\bot\})$	锁的持有者；$\bot$ 表示空闲
Q	$\mathbb{N} \rightharpoonup (\mathcal{P} \times \mathsf{TaskState})$	任务的所属 pipeline、进度或完成结果
n	$\mathbb{N}$	下一次启动使用的新鲜任务编号
C_p	命令	pipeline p 的剩余代码
ρ_p	$\mathsf{Var} \rightharpoonup \mathsf{Val}$	pipeline p 的局部值
η_p	$\mathsf{HVar} \rightharpoonup \mathbb{N}$	句柄变量对应的任务编号

任务状态为

$$\begin{aligned} d \in \mathsf{TaskState} \ ::= \ & \mathrm{Reading}(U, B) \mid \mathrm{Writing}(U, V) \\ & \mid \mathrm{Computing}(k, f, \bar{v}) \mid \mathrm{Done}(v). \end{aligned}$$

U 表示尚未访问的地址集合，B 表示已经读取的结果，V 保存启动时确定的写入值。计算任务保存执行类别、函数和输入值。若 $Q(j) = (p, d)$，则该任务由 pipeline p 启动。读写任务和计算任务都显式记录所属 pipeline。

定义尚未完成的任务集合：

$$\mathrm{Active}_p(Q) = \{ j \mid Q(j) = (p, d) \wedge d \text{ 具有 Reading、Writing 或 Computing 形式} \}.$$

操作句柄的例子是

$$\eta_p(h) = 7, \qquad Q(7) = (p, \mathrm{Done}(5)).$$

执行 `x:=end(h)` 时，先通过 h 查到任务编号 7，再读取任务结果 5，最后把 5 保存到 x。任务编号和计算结果分别存储。

句柄只由 begin 产生，由所属 pipeline 的 end 消费；普通表达式不能复制句柄，也不能把句柄写入共享内存。成功消费后可以重新使用同名句柄变量。已完成但尚未消费的任务可以保留在 Q 中；它们不属于 $\mathrm{Active}_p(Q)$。

初始状态满足

$$Q = \varnothing, \quad n = 0, \quad \eta_p = \varnothing, \quad F(\varepsilon) = 0, \quad L(\ell) = \bot.$$

初始内存 μ_0 与普通局部输入 ρ_p^0 是程序输入的一部分。初始程序的声明、类型和 pipeline 集合必须满足 Γ。

4 带标签的小步操作语义

执行关系写作 $\Gamma \vdash \Sigma \xrightarrow{\alpha} \Sigma'$。以下规则省略固定的 Γ。更新记号 $\Sigma[z \leftarrow v]$ 仅更新指定分量；更新 C_p, ρ_p, η_p 表示更新 $T(p)$ 内对应的字段。顺序执行上下文为

$$E ::= [\,] \mid E; C.$$

规则中 $p \in \mathcal{P}$，当前命令可以处于这样的上下文中。

4.1 局部计算与控制流

$$\frac{C_p = E[x := e] \qquad [\![e]\!]_{\rho_p} = v}{\Sigma \xrightarrow{\mathrm{assign}(p,x,v)} \Sigma[C_p \leftarrow E[\mathsf{skip}],\ \rho_p \leftarrow \rho_p[x \mapsto v]]} \quad (\textsc{Assign})$$

$$\frac{C_p = E[\mathsf{skip}; C]}{\Sigma \xrightarrow{\tau_p} \Sigma[C_p \leftarrow E[C]]} \quad (\textsc{Seq})$$

$$\frac{C_p = E[\mathsf{if}\ e\ \mathsf{then}\ C_1\ \mathsf{else}\ C_2] \qquad [\![e]\!]_{\rho_p} = \mathsf{true}}{\Sigma \xrightarrow{\tau_p} \Sigma[C_p \leftarrow E[C_1]]} \quad (\textsc{If-True})$$

$$\frac{C_p = E[\mathsf{if}\ e\ \mathsf{then}\ C_1\ \mathsf{else}\ C_2] \qquad [\![e]\!]_{\rho_p} = \mathsf{false}}{\Sigma \xrightarrow{\tau_p} \Sigma[C_p \leftarrow E[C_2]]} \quad (\textsc{If-False})$$

$$\frac{C_p = E[\mathsf{while}\ e\ \mathsf{do}\ C]}{\Sigma \xrightarrow{\tau_p} \Sigma[C_p \leftarrow E[\mathsf{if}\ e\ \mathsf{then}\ (C; \mathsf{while}\ e\ \mathsf{do}\ C)\ \mathsf{else}\ \mathsf{skip}]]} \quad (\textsc{While})$$

4.2 参数求值与任务启动

定义任务初始化函数：

$$\begin{aligned}
\mathrm{init}_\rho(\mathsf{read}(e_R)) &= \mathrm{Reading}(R, \varnothing) && \text{当} [\![e_R]\!]_\rho = R, \\
\mathrm{init}_\rho(\mathsf{write}(e_R, e_V)) &= \mathrm{Writing}(R, V) && \text{当} [\![e_R]\!]_\rho = R,\ [\![e_V]\!]_\rho = V, \\
\mathrm{init}_\rho(\mathsf{computation}_{k,f}(\bar{e})) &= \mathrm{Computing}(k, f, \bar{v}) && \text{当} [\![\bar{e}]\!]_\rho = \bar{v}.
\end{aligned}$$

初始化只读取局部环境。实际的共享内存读取由后台规则执行。

令 $\mathrm{Valid}_\Gamma(p, d, \mu)$ 按任务种类定义如下：

$$\begin{aligned}
\mathrm{Valid}_\Gamma(p, \mathrm{Reading}(R, \varnothing), \mu) &\iff R \subseteq \mathrm{dom}(\mu) \wedge \forall a \in R.\ \mathrm{CanRead}(p, a), \\
\mathrm{Valid}_\Gamma(p, \mathrm{Writing}(R, V), \mu) &\iff R \subseteq \mathrm{dom}(\mu) \wedge \mathrm{dom}(V) = R \\
&\quad \wedge \forall a \in R.\ \mathrm{CanWrite}(p, a) \wedge \mathrm{TypeOK}(a, V(a)), \\
\mathrm{Valid}_\Gamma(p, \mathrm{Computing}(k, f, \bar{v}), \mu) &\iff \mathrm{kind}(p) = k \wedge \bar{v} \in D_f \\
&\quad \wedge \mathrm{Supports}(p, f, \mathrm{types}(\bar{v})).
\end{aligned}$$

这里的区域和值还须具有相应语法要求的类型，TypeOK 由地址的值类型声明决定。

$$\frac{C_p = E[h := \mathsf{begin}(o)] \qquad h \notin \mathrm{dom}(\eta_p) \qquad d = \mathrm{init}_{\rho_p}(o) \qquad \mathrm{Valid}_\Gamma(p, d, \mu)}{\Sigma \xrightarrow{\mathrm{begin}(p,n,o)} \Sigma'} \quad (\textsc{Begin})$$

$$\Sigma' = \Sigma[C_p \leftarrow E[\mathsf{skip}],\ \eta_p \leftarrow \eta_p[h \mapsto n],\ Q \leftarrow Q[n \mapsto (p, d)],\ n \leftarrow n + 1]$$

所有已创建的任务编号都小于 n，因此新编号不会复用。该规则没有要求 $\mathrm{Active}_p(Q) = \varnothing$，后续操作可以在已有任务完成之前启动。输入值在此处保存；之后的局部变量赋值不会改变已经启动的任务参数。

4.3 后台读写与完成

$$\frac{Q(j) = (p, \mathrm{Reading}(U, B)) \qquad a \in U \qquad \mu(a) = v}{\Sigma \xrightarrow{\mathrm{read}(p,j,a,v)} \Sigma[Q \leftarrow Q[j \mapsto (p, \mathrm{Reading}(U \setminus \{a\}, B[a \mapsto v]))]]} \quad (\textsc{Read})$$

$$\frac{Q(j) = (p, \mathrm{Writing}(U, V)) \qquad a \in U}{\Sigma \xrightarrow{\mathrm{write}(p,j,a,V(a))} \Sigma'} \quad (\textsc{Write})$$

$$\Sigma' = \Sigma[\mu \leftarrow \mu[a \mapsto V(a)],\ Q \leftarrow Q[j \mapsto (p, \mathrm{Writing}(U \setminus \{a\}, V))]]$$

每个步骤访问一个存储单元。区域内尚未访问的任意地址都可以作为下一步；不同任务的访问可以交错。完整区域的读取或写入具有多个步骤，读取结果可以包含来自不同写入阶段的值。

$$\frac{Q(j) = (p, \mathrm{Reading}(\varnothing, B))}{\Sigma \xrightarrow{\mathrm{finish}(p,j)} \Sigma[Q \leftarrow Q[j \mapsto (p, \mathrm{Done}(B))]]} \quad (\textsc{Finish-Read})$$

$$\frac{Q(j) = (p, \mathrm{Writing}(\varnothing, V))}{\Sigma \xrightarrow{\mathrm{finish}(p,j)} \Sigma[Q \leftarrow Q[j \mapsto (p, \mathrm{Done}(\mathsf{unit}))]]} \quad (\textsc{Finish-Write})$$

$$\frac{Q(j) = (p, \mathrm{Computing}(k, f, \bar{v})) \qquad f(\bar{v}) = v}{\Sigma \xrightarrow{\mathrm{finish}(p,j)} \Sigma[Q \leftarrow Q[j \mapsto (p, \mathrm{Done}(v))]]} \quad (\textsc{Finish-Compute})$$

纯计算从 begin 到 finish 具有一个可与其他任务重叠的执行区间。其内部数值运算由 f 给出；该完成步骤只更新 Q。本模型不赋予这些步骤具体时长。

4.4 等待并取得结果

$$\frac{C_p = E[x := \mathsf{end}(h)] \qquad \eta_p(h) = j \qquad Q(j) = (p, \mathrm{Done}(v))}{\Sigma \xrightarrow{\mathrm{join}(p,j)} \Sigma'} \quad (\textsc{End})$$

$$\Sigma' = \Sigma[C_p \leftarrow E[\mathsf{skip}],\ \rho_p \leftarrow \rho_p[x \mapsto v],\ \eta_p \leftarrow \eta_p \setminus \{h\},\ Q \leftarrow Q \setminus \{j\}]$$

任务尚未完成时，End 规则的前提不成立，pipeline 停留在当前命令。其他 pipeline 和后台任务仍然可以执行。任务可以在 pipeline 到达 end 之前完成，此时取得结果的步骤直接可用。

5 事件与锁的同步语义

5.1 二值事件

$$\frac{C_p = E[\mathsf{setflag}(\varepsilon)] \qquad \mathrm{MaySet}(p,\varepsilon) \qquad \mathrm{Active}_p(Q) = \varnothing}{\Sigma \xrightarrow{\mathrm{set}(p,\varepsilon)} \Sigma[C_p \leftarrow E[\mathsf{skip}],\ F \leftarrow F[\varepsilon \mapsto 1]]} \quad (\textsc{SetFlag})$$

$$\frac{C_p = E[\mathsf{waitflag}(\varepsilon)] \qquad \mathrm{MayWait}(p,\varepsilon) \qquad F(\varepsilon) = 1}{\Sigma \xrightarrow{\mathrm{wait}(p,\varepsilon)} \Sigma[C_p \leftarrow E[\mathsf{skip}],\ F \leftarrow F[\varepsilon \mapsto 0]]} \quad (\textsc{WaitFlag})$$

setflag 等待源 pipeline 的全部前序任务完成。已完成且已经通过 end 消费的任务也满足这一完成要求。waitflag 在标志为 0 时阻塞后续命令；成功时消费标志并使其恢复为 0。目标 pipeline 此前已经启动的任务仍然可以执行。重复设置尚未消费的标志得到 1，因而不会累积通知数量。

5.2 互斥锁

$$\frac{C_p = E[\mathsf{lock}(\ell)] \qquad p \in \mathrm{Participants}(\ell) \qquad L(\ell) = \bot}{\Sigma \xrightarrow{\mathrm{acquire}(p,\ell)} \Sigma[C_p \leftarrow E[\mathsf{skip}],\ L \leftarrow L[\ell \mapsto p]]} \quad (\textsc{Lock})$$

$$\frac{C_p = E[\mathsf{unlock}(\ell)] \qquad p \in \mathrm{Participants}(\ell) \qquad L(\ell) = p}{\Sigma \xrightarrow{\mathrm{release}(p,\ell)} \Sigma[C_p \leftarrow E[\mathsf{skip}],\ L \leftarrow L[\ell \mapsto \bot]]} \quad (\textsc{Unlock})$$

锁采用非递归互斥语义。锁被持有时，新的获取操作等待；当前持有者再次请求同一把锁也等待。释放锁必须由持有者执行。锁的获取和释放建立 acquire/release 同步关系，具体 happens-before 定义见第 6.3 节。

unlock 的前提仅检查所有权及参与权限。临界区内启动的异步任务需要通过 end 或已建立的其他完成依赖，保证访问完成后再释放锁。例如

```
lock(l);
h := begin(write(R, V));
unit := end(h);
unlock(l);
```

使任务中的全部写入先于锁释放。

5.3 有序锁的策略实例

需要按给定参与者顺序放行时，为该锁使用状态 $L(\ell) = (owner, r)$，初值为 $(\bot, 0)$，并声明序列 $w_\ell : \mathbb{N} \to \mathrm{Participants}(\ell)$。锁的声明及参与权限检查继续适用。Lock 的所有权前提和更新改为

$$owner = \bot \wedge p = w_\ell(r), \qquad L(\ell) \leftarrow (p, r),$$

Unlock 的前提要求当前所有者为 p，更新为

$$L(\ell) \leftarrow (\bot, r+1).$$

例如轮转序列满足 $w_\ell(r) = p_{r \bmod m}$。这一策略显式保留访问次序带来的等待依赖。具体硬件的 block/subblock 编号、跳过临界区时的处理和内存同步要求，需要由其翻译规则给出。其余章节和执行例子均使用普通互斥锁的状态类型。

6 错误、全局执行与并发性质

6.1 等待条件与错误状态

end 对未完成任务的等待、waitflag 对零标志的等待、lock 对已持有锁的等待，以及 setflag 对前序任务完成的等待，都通过相应规则暂时不可用表达。

令 $\mathrm{Bad}_\Gamma(p, a, \Sigma)$ 表示当前命令存在以下错误之一：

- 普通表达式引用未定义变量、违反类型要求，或纯表达式运算未定义；条件表达式的结果必须为布尔值。
- begin 使用已有活动句柄变量，或初始化任务不满足 Valid_Γ。
- end 使用不存在的句柄、查不到任务，或句柄指向其他 pipeline 的任务。
- 事件未声明、事件端点与执行 pipeline 不匹配，或同步域声明不合法。
- 锁未声明、pipeline 不属于允许参与者，或执行 unlock 的 pipeline 未持有该锁。

条件求值的错误同样通过以下规则报告；此处 a 也可代表待求值的条件命令。

$$\frac{C_p = E[a] \qquad \mathrm{Bad}_\Gamma(p, a, \Sigma)}{\Sigma \xrightarrow{\mathrm{error}(p,a)} \mathsf{Error}} \quad (\textsc{Error})$$

Error 没有后继状态。已完成但尚未消费的句柄也占用其句柄变量名。用于等待的条件不会单独触发 Bad_Γ。

6.2 全局交错与轨迹

所有规则直接作用于同一全局状态。全局关系是控制步骤、后台任务步骤和错误步骤的并集：

$$\rightarrow = \bigcup_{p \in \mathcal{P}} \rightarrow_{\mathrm{control},p} \ \cup \bigcup_{j \in \mathrm{dom}(Q)} \rightarrow_{\mathrm{task},j} \ \cup \ \rightarrow_{\mathrm{error}} .$$

每一步非确定地选择一个满足前提的规则。同一 pipeline 的控制命令按照程序顺序执行，多个未完成任务可以共存；当前抽象允许它们以任意顺序完成。不同 pipeline 的任务也可以交错执行。

初始状态集合记为 $\mathrm{Init}_\Gamma(P)$。程序语义是全部最大轨迹：

$$[\![P]\!]_\Gamma = \left\{ \Sigma_0 \xrightarrow{\alpha_0} \Sigma_1 \xrightarrow{\alpha_1} \cdots \;\middle|\; \Sigma_0 \in \mathrm{Init}_\Gamma(P) \right\}_{\mathrm{maximal}} .$$

最大轨迹是无限轨迹，或没有后继步骤的有限轨迹。安全性质对全部轨迹量化。活性论证可以进一步要求弱公平：持续可执行的 pipeline 或任务最终获得执行机会。互斥锁的无饥饿结论还需要相应仲裁公平性。

6.3 Happens-before 与数据竞争

对轨迹中每次转移赋予唯一事件标识。定义以下关系：

1. po：同一 pipeline 的相邻控制事件按执行顺序连接，包含 begin、join、同步操作、局部赋值和控制流步骤。
2. $task$：一个任务的 begin 连接其第一个后台步骤，该任务的相邻后台步骤依次连接，直至 finish。
3. $join$：任务的 finish 连接取得该结果的 join。
4. $drain$：源 pipeline 在某次 setflag 之前启动的每个任务，其 finish 连接该次 setflag。此关系根据轨迹中的启动历史定义，包含已经从 Q 移除的任务。
5. $flag$：设置事件连接到消费它的成功等待事件，方向为 $\mathrm{set}(\varepsilon) \to \mathrm{wait}(\varepsilon)$。设置时可以在分析记录中保存最近一次设置事件的标识；等待成功时读取并清除它。这些分析标识不参与程序求值。
6. $mutex$：同一把锁的一次释放连接其后第一次成功获取。

定义

$$hb = (po \cup task \cup join \cup drain \cup flag \cup mutex)^{+}.$$

两个独立后台任务在某条轨迹中先后完成，这一时间先后关系本身不会增加 hb 边。操作间的同步依赖由上述关系确定。

对访问事件 a, b，定义

$$\begin{aligned} \mathrm{Conflict}(a,b) &\iff \mathrm{loc}(a) = \mathrm{loc}(b) \\ &\quad \wedge (\mathrm{isWrite}(a) \vee \mathrm{isWrite}(b)), \\ \mathrm{Race}(a,b) &\iff a \neq b \wedge \mathrm{Conflict}(a,b) \\ &\quad \wedge \neg(a \ hb \ b) \wedge \neg(b \ hb \ a). \end{aligned}$$

单个存储单元的原子访问仍然可能与另一任务的数据访问构成数据竞争。这一判定也覆盖同一 pipeline 内缺少完成依赖的两个异步任务。程序数据竞争自由表示其全部轨迹都不存在这样的事件对。

6.4 终止与死锁

$$\mathrm{Terminal}(\Sigma) \iff (\forall p \in \mathcal{P}.\ C_p = \mathsf{skip}) \wedge (\forall p \in \mathcal{P}.\ \mathrm{Active}_p(Q) = \varnothing).$$

未消费的已完成任务允许保留；正常终止只要求全部代码和后台操作完成。

$$\begin{aligned} \mathrm{Deadlocked}(\Sigma) &\iff \Sigma \neq \mathsf{Error} \wedge \neg\,\mathrm{Terminal}(\Sigma) \\ &\quad \wedge \nexists \alpha, \Sigma'.\ \Sigma \xrightarrow{\alpha} \Sigma'. \end{aligned}$$

这一定义描述全局死锁。存在后台任务可继续执行时，系统仍有后继步骤。某些 pipeline 永久等待而其他 pipeline 无限运行，需要通过活性性质另外描述。

7 硬件对应条件与语义依据

本语言采用原子存储单元上的顺序一致性交错内存模型，显式描述任务启动、完成、事件消费和锁所有权。有限队列容量、实际运算时延、缓存维护及弱内存传播需要由具体硬件实例的语义给出。同一 pipeline 的内部阶段顺序也属于需要核查的硬件约束。

将具体程序 P 翻译为 core language 程序 $T(P)$ 时，需要保持实际访存、地址别名、执行类别、pipeline 控制关系、同步作用域和异步完成依赖。若使用本语言证明硬件安全性质，一项需要建立的义务是

$$\alpha(\mathrm{Traces}_{\mathrm{hardware}}(P)) \subseteq \mathrm{Traces}_{\mathrm{core}}(T(P)),$$

其中 α 投影硬件事件。把数据竞争或无死锁结论传递到硬件时，还分别需要冲突与同步关系保持，以及具体执行的进展性质。对应的 Rocq 定义位于 `rocq/CoreLanguage/`，包含语法、执行状态、带标签的小步关系、最大执行轨迹、happens-before、数据竞争和可选顺序锁策略。其中给出了任务编号新鲜性、计算类别检查等性质证明及具体执行推导。硬件对应定理仍需根据具体目标建立。

核内 setflag/waitflag 的二值消费行为与源 pipeline 完成条件参照 Ascend 文档 [1, 2]。HIVM 同步操作携带源 pipeline、目标 pipeline 和事件编号，仓库中可核查的定义位于 `vendor/ascendnpu-ir/upstream/` 下的 `bishengir/include/bishengir/Dialect/HIVM/IR/HIVMSynchronizationOps.td`；运行条件也见 `docs/source/zh_cn/developer_guide/features/auto_sync.md`。该参考目录记录的上游版本为 `22e3d51df4642fb0f74060af36f5c5c80be80d08`。

锁的 acquire/release 含义参考 Triton 原子操作接口提供的内存语义。Triton LayerNorm 示例在临界区释放锁之前执行线程同步，体现了完成依赖的要求 [3, 4, 5]。本文的 lock/unlock 是抽象原语；具体原子指令序列及其作用域由目标语言翻译确定。

参考文献

[1] Ascend C，*SetFlag/WaitFlag*。https://asc.gitcode.com/api/SIMD-API/basic_api/sync_control/intra_core_sync/SetFlag-WaitFlag.html。

[2] Ascend C，*Basic Architecture*，CANN 9.0。https://www.hiascend.com/document/detail/en/CANNCommunityEdition/900/programug/Ascendcopdevg/atlas_ascendc_10_0008.html。

[3] Triton，*triton.language.atomic_cas*。https://triton-lang.org/main/python-api/generated/triton.language.atomic_cas.html。

[4] Triton，*triton.language.atomic_xchg*。https://triton-lang.org/main/python-api/generated/triton.language.atomic_xchg.html。

[5] Triton，*Layer Normalization*。https://triton-lang.org/main/getting-started/tutorials/05-layer-norm.html。

8 具体执行例子

例子中的 `read` 返回地址值映射，`write` 接收这样的映射。代码里的 `unit` 是普通变量名，用于接收写入任务返回的 unit。每个轨迹表列出实际产生状态变化的语义步骤；控制位置的 skip 消除步骤不逐项列出。等待条件不满足时没有成功转移，表格会直接注明等待位置。各例子的执行类别和函数支持关系由各自声明确定。

8.1 操作句柄：启动、后台完成与取得结果

取 $\mathcal{P} = \{p_v\}$、$\mathrm{kind}(p_v) = \mathsf{Vector}$，并声明 p_v 支持整数加法 $\mathrm{add}(a, b) = a + b$。初始局部映射和任务表为空，$n = 0$。

```
parallel {
  pv:
    h := begin(computation[Vector, add](2, 3));
    y := 10;
    x := end(h);
    z := x + y;
}
```

记启动时的任务内容为

$$d_0 = \mathrm{Computing}(\mathsf{Vector}, \mathrm{add}, [2, 3]).$$

步骤	执行动作	局部值与句柄	任务状态
1	启动加法，分配编号 0	$\eta_{p_v}(h) = 0$	$Q(0) = (p_v, d_0)$
2	执行 $y := 10$	$\rho_{p_v}(y) = 10$，$\eta_{p_v}(h) = 0$	任务 0 尚未完成
3	后台完成加法	pipeline 位于 $x := \mathsf{end}(h)$	$Q(0) = (p_v, \mathrm{Done}(5))$
4	取得任务结果	$\rho_{p_v}(x) = 5$，移除句柄 h	移除 $Q(0)$
5	执行 $z := x + y$	$\rho_{p_v}(z) = 15$	$Q = \varnothing$

步骤 2 之后，若控制步骤尝试执行 end，其完成前提尚未满足。后台步骤 3 可以执行并使 end 可用。步骤 4 的查询过程是

$$\eta_{p_v}(h) = 0, \qquad Q(0) = (p_v, \mathrm{Done}(5)), \qquad \rho'_{p_v}(x) = 5.$$

编号 0 标识这一次任务；值 5 是任务计算结果。整个例子的共享内存保持原值。

8.2 同一 pipeline 的两个在途任务

取 $\mathcal{P} = \{p_m\}$、$\mathrm{kind}(p_m) = \mathsf{MTE}$，允许访问地址 a, b，初始 $\mu(a) = \mu(b) = 0$。两个写入区域不相交。

```
parallel {
  pm:
```

```
    h1 := begin(write({a}, {a: 11}));
    h2 := begin(write({b}, {b: 22}));
    unit := end(h1);
    unit := end(h2);
}
```

步骤	动作	$\mu(a)$	$\mu(b)$	任务与控制状态
1	$\mathrm{begin}(p_m, 0)$	0	0	$h_1 \mapsto 0$，任务 0 写入 a
2	$\mathrm{begin}(p_m, 1)$	0	0	$h_2 \mapsto 1$，两个任务均未完成
3	任务 1 写入 b	0	22	任务 1 的剩余地址集合为空
4	任务 1 完成	0	22	$Q(1) = (p_m, \mathrm{Done}(\mathsf{unit}))$
5	任务 0 写入 a	11	22	pipeline 仍在等待 h_1
6	任务 0 完成	11	22	两个任务都已完成
7	$\mathrm{join}(p_m, 0)$	11	22	消费 h_1
8	$\mathrm{join}(p_m, 1)$	11	22	消费 h_2，程序终止

这条轨迹满足同一 pipeline 的启动顺序，同时允许任务 1 先完成。任务 0 的执行区间包含任务 1 的启动与完成。这是本语言允许的抽象调度；具体 pipeline 对完成顺序的限制需要通过硬件对应条件核查。

8.3 Cube 与 Vector 的类别检查和并行计算

声明

$$\mathcal{P} = \{p_c, p_v\}, \qquad \mathrm{kind}(p_c) = \mathsf{Cube}, \qquad \mathrm{kind}(p_v) = \mathsf{Vector}.$$

p_c 支持 2×2 整数矩阵乘法，p_v 支持长度为 2 的整数向量加法。初始局部值为

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, \qquad X = (10, 20), \quad Y = (1, 2).$$

A, B 位于 ρ_{p_c}，X, Y 位于 ρ_{p_v}。

```
parallel {
  pc:
    hc := begin(computation[Cube, matmul](A, B));
    C := end(hc);

  pv:
    hv := begin(computation[Vector, vadd](X, Y));
    Z := end(hv);
}
```

一次允许的执行是：

步骤	动作	结果状态
1	Cube pipeline 启动任务 0	$Q(0) = (p_c, \text{Computing}(\mathsf{Cube}, \text{matmul}, [A, B]))$
2	Vector pipeline 启动任务 1	$Q(1) = (p_v, \text{Computing}(\mathsf{Vector}, \text{vadd}, [X, Y]))$
3	Vector 任务完成	$Q(1) = (p_v, \text{Done}((11, 22)))$
4	Vector pipeline 取得结果	$\rho_{p_v}(Z) = (11, 22)$
5	Cube 任务完成	$Q(0) = (p_c, \text{Done}(AB))$
6	Cube pipeline 取得结果	$\rho_{p_c}(C) = AB$

其中

$$AB = \begin{pmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

步骤 2 之后，Cube 和 Vector 任务同时处于未完成状态。各任务分别检查

$$\text{kind}(p_c) = \mathsf{Cube}, \qquad \text{kind}(p_v) = \mathsf{Vector},$$

以及对应的 Supports 条件。在另一个输入状态中，令 ρ_{p_v} 也包含上述矩阵 A, B，并声明 p_v 支持该数学函数。在 p_v 上启动 `computation[Cube,matmul](A,B)` 仍然使 $\text{kind}(p_v) = \mathsf{Cube}$ 这一类别前提失败，进入 Error，且不会创建任务。本例全部计算使用局部值，μ 保持不变。

8.4 事件等待前序异步写入完成

声明 $\text{kind}(p_m) = \mathsf{MTE}$、$\text{kind}(p_v) = \mathsf{Vector}$，事件 $\varepsilon = (s, p_m, p_v, 0)$ 初值为 0，两个 pipeline 均有访问 a 的相应权限，初始 $\mu(a) = 0$。Vector pipeline 支持整数加法。

```
parallel {
  pm:
    hw := begin(write({a}, {a: 41}));
    setflag(s, pm, pv, 0);

  pv:
    waitflag(s, pm, pv, 0);
    hr := begin(read({a}));
    B := end(hr);
    hc := begin(computation[Vector, add](B[a], 1));
    result := end(hc);
}
```

步骤	动作	$\mu(a)$	$F(\varepsilon)$
1	MTE 启动写入任务 0	0	0
2	任务 0 写入 a	41	0
3	任务 0 完成	41	0
4	MTE 执行 setflag	41	1
5	Vector 执行 waitflag，消费标志	41	0
6	Vector 启动读取任务 1	41	0
7	任务 1 读取 a，得到 41	41	0
8	任务 1 完成	41	0
9	Vector 取得 $B = \{a \mapsto 41\}$	41	0
10	Vector 启动加法任务 2	41	0
11	任务 2 完成，得到 42	41	0
12	Vector 取得 $result = 42$	41	0

步骤 1 之后，MTE 位于 setflag，但 $\mathrm{Active}_{p_m}(Q) = \{0\}$，因此必须等待步骤 3 完成。Vector 的 waitflag 在步骤 4 之前也不可用。写入任务 0 的句柄可以保持未消费：它已完成，因而不妨碍事件设置或程序终止。对应的 happens-before 链为

$$\mathrm{write}(0, a, 41)\ hb\ \mathrm{finish}(0)\ hb\ \mathrm{set}(\varepsilon)\ hb\ \mathrm{wait}(\varepsilon)\ hb\ \mathrm{read}(1, a, 41).$$

此处省略标签中的 pipeline 标识，仅用于缩短公式。

8.5 临界区中的异步写入

两个 pipeline 共享锁 ℓ 和地址 a，拥有相应读写权限。初始 $L(\ell) = \bot$、$\mu(a) = 0$。

```
parallel {
  p1:
    lock(l);
    h := begin(write({a}, {a: 5}));
    unit := end(h);
    unlock(l);

  p2:
    lock(l);
    h := begin(read({a}));
    B := end(h);
    unlock(l);
}
```

选择 p_1 首先获得锁的执行：

步骤	动作	锁状态
1	p_1 获取锁	$L(\ell) = p_1$
2	p_1 启动写入任务 0	$L(\ell) = p_1$
3	任务 0 写入 a，$\mu(a) = 5$	$L(\ell) = p_1$
4	任务 0 完成	$L(\ell) = p_1$
5	p_1 取得完成结果	$L(\ell) = p_1$
6	p_1 释放锁	$L(\ell) = \bot$
7	p_2 获取锁	$L(\ell) = p_2$
8	p_2 启动读取任务 1	$L(\ell) = p_2$
9	任务 1 读取 a，得到 5	$L(\ell) = p_2$
10	任务 1 完成	$L(\ell) = p_2$
11	p_2 取得 $B = \{a \mapsto 5\}$	$L(\ell) = p_2$
12	p_2 释放锁	$L(\ell) = \bot$

在步骤 1–5 期间，p_2 的获取规则不可用。写入访问经由任务完成、join、锁释放与再次获取，先于读取访问。该程序也允许 p_2 首先获得锁，此时它读取初值 0；两种顺序都受到互斥保护。

若某程序在启动写入后立即释放锁，再等待该写入，则其允许的事件前缀为

$$\begin{aligned}&\mathrm{acquire}(p_1, \ell),\ \mathrm{begin}(p_1, 0),\ \mathrm{release}(p_1, \ell),\\ &\mathrm{acquire}(p_2, \ell),\ \mathrm{begin}(p_2, 1),\ \mathrm{read}(p_2, 1, a, 0),\ \mathrm{write}(p_1, 0, a, 5).\end{aligned}$$

任务 0 的写入缺少通向第一次锁释放的完成依赖，两个访问构成数据竞争。

8.6 二值事件复用与循环等待

对于同一个事件 ε，存在如下执行顺序：

动作	初始	set	set	wait	再次等待
$F(\varepsilon)$	0	1	1	0	0

两次设置之间没有消费时，第二次等待仍需要后续设置才能成功。若采用设置、消费、设置、消费的顺序，两次等待均可完成。

下面的程序声明两个独立事件 $\varepsilon_{pq} = (s, p, q, 0)$ 与 $\varepsilon_{qp} = (s, q, p, 0)$，它们初始均为 0：

```
parallel {
  p:
    waitflag(s, q, p, 0);
    setflag(s, p, q, 0);

  q:
    waitflag(s, p, q, 0);
    setflag(s, q, p, 0);
}
```

初始状态满足

$$Q = \varnothing, \qquad F(\varepsilon_{pq}) = F(\varepsilon_{qp}) = 0.$$

两个 pipeline 的当前命令都是等待，两个等待前提均不成立，后台任务集合为空，也不存在错误步骤。剩余代码尚未完成，因此该初始状态满足 $\mathrm{Deadlocked}(\Sigma_0)$。